УКРАЇНСЬКИЙ ДЕРЖАВНИЙ УНІВЕРСИТЕТ НАУКИ І ТЕХНОЛОГІЙ
ННІ “ПРИДНІПРОВСЬКА ДЕРЖАВНА АКАДЕМІЯ БУДІВНИЦТВА ТА АРХІТЕКТУРИ”
Кафедра комп’ютерних наук, інформаційних технологій та прикладної математики

Лабораторна робота №3

З курсу “Нейронні мережі”

Тема: “Апроксимація лінійної функції та класифікація образів за допомогою нейронної мережі”

Варіант 11

Студента третього курсу
Групи КН-23
спеціальності «Комп’ютерні науки»
Виконав: Маслюков І. С.
Перевірив: ст.в. Плахтій Є. Г.

м. Дніпро 2026

ЗМІСТ

Теоретична частина **3**
Частина 1 ..... **5**
Перше завдання 5
Виконання 1 5
Друге завдання 6
Виконання 2 7
Висновки до першої частини 11
Частина 2 ..... **13**
Завдання першого блоку (Апроксимація функції) 13
Виконання 1 14
Завдання другого блоку 16
Виконання 2 16
Висновки **19**
Контрольні питання **21**

Теоретична частина

Даний етап розвитку нейромережевих технологій характеризується переходом від найпростіших лінійних моделей до складних багатошарових та радіальних структур, здатних вирішувати задачі класифікації та апроксимації у нелінійних середовищах. Для подолання обмежень, властивих лінійним нейронним мережам, застосовуються багатошарові мережі прямого поширення сигналів, архітектура яких включає один або декілька прихованих шарів. Встановлено, що мережа навіть з одним прихованим шаром володіє достатньою математичною гнучкістю для здійснення будь-якого нелінійного перетворення типу «вхід-вихід». Процес проектування таких систем передбачає визначення оптимальної топології, де кількість нейронів у вихідному шарі жорстко обмежена умовами конкретної задачі, а параметри прихованих шарів часто обираються на основі досвіду розробника або за допомогою евристичних правил. Одним із найбільш розповсюджених підходів є встановлення кількості нейронів прихованого шару на рівні половини сумарної кількості входів та виходів мережі.

Навчання багатошарових мереж прямого поширення базується на ітеративному застосуванні методу зворотного поширення помилки. У середовищі MATLAB для реалізації цього процесу використовується функція feedforwardnet, яка дозволяє гнучко налаштовувати кількість шарів та вибирати алгоритм навчання. Серед доступних модифікацій градієнтного спуску особливе місце посідає алгоритм Левенберга-Марквардта (trainlm), який забезпечує максимальну швидкодію за рахунок поєднання властивостей методу Ньютона та класичного градієнтного спуску. Незважаючи на високі вимоги до обсягу оперативної пам'яті, даний алгоритм є стандартом де-факто для задач апроксимації середньої складності. Важливим аспектом є контроль параметрів навчання, таких як цільова помилка (goal), максимальна кількість

епох та градієнт, що дозволяє уникнути перенавчання та забезпечити стабільну роботу моделі на нових даних.

Альтернативним підходом до побудови нелінійних моделей є використання нейронних мереж із радіальними базисними активаційними функціями (RBF). На відміну від багатошарових персептронів, радіальні мережі мають двошарову структуру, де перший шар складається з радіально-симетричних нейронів, а другий виконує роль лінійного суматора. Для автоматичного синтезу архітектури таких мереж у MATLAB призначена функція newrb, яка реалізує ітераційний алгоритм додавання нейронів у прихований шар до моменту досягнення заданої точності апроксимації. Ключовим параметром тут є ширина базисної функції (spread), від якої залежить гладкість отриманої поверхні та здатність мережі до інтерполяції значень у точках, що не входили до навчальної вибірки.

Окремим класом радіальних структур є імовірнісні нейронні мережі (PNN), що спеціалізуються на задачах класифікації образів. Архітектурна особливість PNN полягає у використанні конкурентної активаційної функції у вихідному шарі, яка автоматично ідентифікує нейрон із максимальним сигналом і встановлює його значення рівним одиниці, обнуляючи всі інші виходи. Це забезпечує чітку нормалізацію результатів класифікації та високу стійкість системи до зашумлення вхідних даних. Кількість нейронів у прихованому шарі такої мережі завжди дорівнює кількості вхідних векторів у навчальній послідовності, що дозволяє реалізувати механізм швидкого навчання без тривалого ітераційного підбору вагових коефіцієнтів. Поєднання розглянутих архітектур дозволяє створювати комплексні інтелектуальні системи, здатні до ефективного розпізнавання складних образів та ідентифікації прихованих закономірностей у динамічних процесах.

Частина 1

Перше завдання

1. Визначити топологію та створити нейромережу прямого поширення для моделювання логічного елемента

2. Підготувати навчальну вибірку та провести тренування моделі, обравши один із варіантів алгоритму зворотного поширення помилки.

3. Здійснити перевірку (тестування) отриманої нейронної мережі на коректність роботи.

Виконання 1

— Для навчання використаємо стандартний та найшвидший алгоритм Левенберга-Марквардта (trainlm)

```
% 1. Підготовка навчальної вибірки (XOR)
% Вхідні дані (стовпці - це окремі образи)
X = [0 1 0 1;
     0 0 1 1];
% Цільові значення
T = [0 1 1 0];

% 2. Створення багатошарової мережі
% Прихований шар із 3 нейронами, функція навчання - 'trainlm'
net = feedforwardnet(3, 'trainlm');

% Налаштування параметрів навчання
net.trainParam.epochs = 1000; % Максимальна кількість епох
net.trainParam.goal = 1e-5;    % Цільова похибка
net.divideFcn = '';           % Використовувати всі дані для навчання

% 3. Тренування моделі
net = train(net, X, T);

% 4. Тестування мережі
test = [0 0 1 1;
        1 0 0 1];
Y = net(test);
disp('Результати роботи мережі (XOR):');
disp(Y);
disp('Округлені результати:');
disp(round(Y));
```

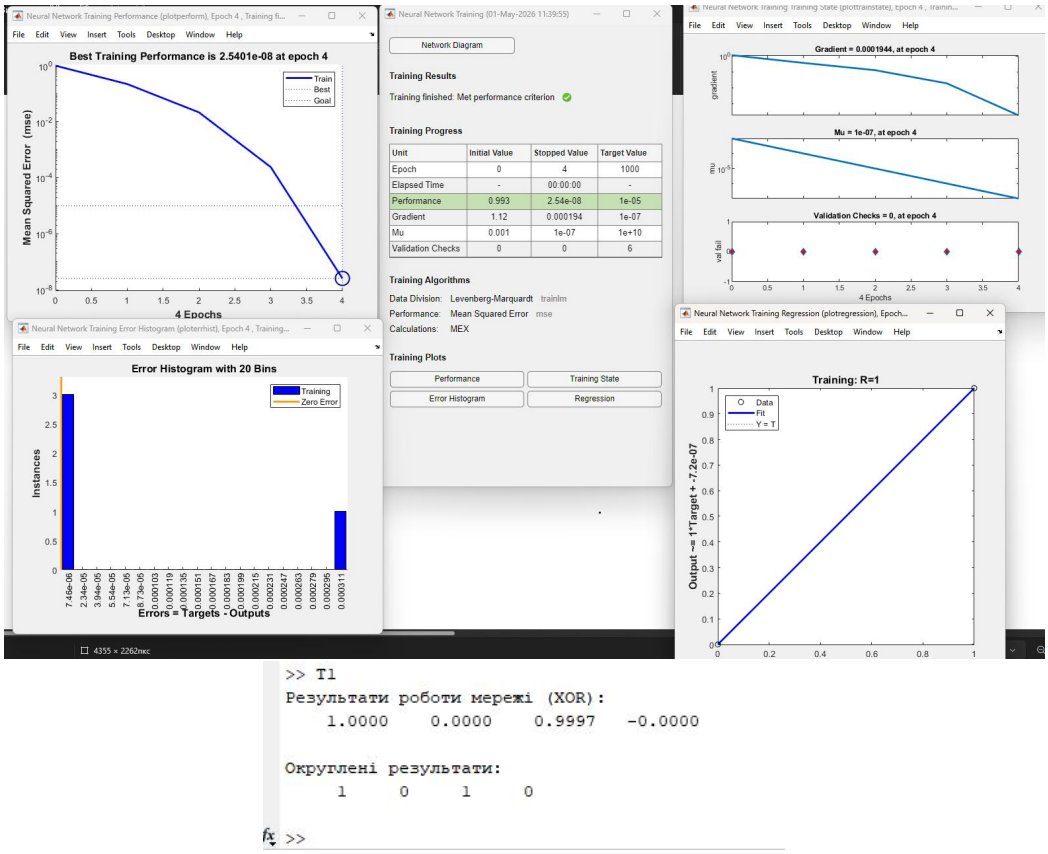
Рис. 1.1 — Код

Функція feedforwardnet створює мережу, де один прихований шар здатний здійснити будь-яке нелінійне перетворення.

За алгоритм навчання ми обрали trainlm, оскільки він забезпечує максимальну швидкодію в порівнянні з іншими модифікаціями методу зворотного поширення помилки.

Критерій зупинки навчання завершиться автоматично при досягненні заданої помилки (goal) або ліміту епох (epochs).

.




```
>> T1
Результати роботи мережі (XOR):
    1.0000    0.0000    0.9997   -0.0000

Округлені результати:
     1     0     1     0

>>
```

Рис. 1.2 — Результати виконання

— Як бачимо мережа навчилася і видає правильні результати

Друге завдання

1. Спроєктувати та сформувати архітектуру мережі прямого поширення для ідентифікації латинських літер. Система має видавати одиницю на тому виході, що відповідає порядковому номеру символу в алфавіті, за умови подачі на вхід бінарного коду цієї літери.

2. Сформувати набір даних та навчити розроблену нейромережу розпізнаванню символів.

3. Проаналізувати стійкість навченої моделі до перешкод, використовуючи функцію randn для додавання випадкового шуму (з інтервалу (–1 1), нульовим середнім та відхиленням <=0,5) до вхідних векторів. Вектор шуму додається до відповідного вектора-стовпця масиву alphabet.

4. Побудувати графічну залежність вихідної помилки від інтенсивності шуму. Для цього необхідно:

• Згенерувати шумові вектори з кроком відхилення 0,05 (від 0 до 0,5).

• Для кожного кроку підготувати по 10 зашумлених зразків для кожної літери.

• Отримати відгуки мережі на ці дані та розрахувати середню сумарну помилку (як евклідову норму різниці з еталоном).

• Візуалізувати отримані результати у вигляді графіка.

Виконання 2

Для розпізнавання 26 літер латинського алфавіту ми використаємо функцію prprob, яка генерує матрицю шаблонів 7 • 5 (35 вхідних сигналів) та відповідні цільові вектори (26 виходів).

Розрахунок архітектури:

Входи N: 35.

Виходи N: 26.

Прихований шар: За евристичним правилом (35 + 26) / 2 = 30.5. Округлюємо до 31 нейрона.

Дані візьмемо з бібліотеки “prprob”.

```
% --- Крок 1: Підготовка даних ---
prprob_my;


% --- Крок 2: Створення та навчання мережі ---
net = feedforwardnet(31, 'trainlm'); % 31 нейрон у прихованому шарі
net.trainParam.goal = 1e-6;          % Цільова помилка
net.divideFcn = '';                  % Використовувати всі дані
net = train(net, alphabet, targets); % Навчання на "чистих" даних

% --- Крок 3: Дослідження стійкості до шуму (п. 6-7) ---
noise_levels = 0:0.05:0.5;           % Рівні шуму від 0 до 0.5
mean_errors = zeros(size(noise_levels)); % Масив для збереження помилок

for i = 1:length(noise_levels)
    std_dev = noise_levels(i);       % Поточне відхилення шуму
    total_error = 0;

    for char_idx = 1:26              % Для кожної з 26 літер
        for trial = 1:10             % По 10 зашумлених зразків
            % Додавання шуму за допомогою randn
            noisy_input = alphabet(:, char_idx) + std_dev * randn(35, 1);

            % Отримання виходу мережі
            y_out = net(noisy_input);

            % Розрахунок помилки як евклідової норми різниці
            total_error = total_error + norm(targets(:, char_idx) - y_out);
        end
    end
    % Обчислення середньої сумарної помилки для рівня шуму
    mean_errors(i) = total_error / (26 * 10);
end

% --- Крок 4: Візуалізація результатів ---
figure;
plot(noise_levels, mean_errors, 'r-s', 'LineWidth', 2);
title('Залежність помилки розпізнавання від рівня шуму');
xlabel('Інтенсивність шуму (Standard Deviation)');
ylabel('Середня сумарна помилка (Euclidean Norm)');
grid on;
```

Рис. 1.3 — Код мережі

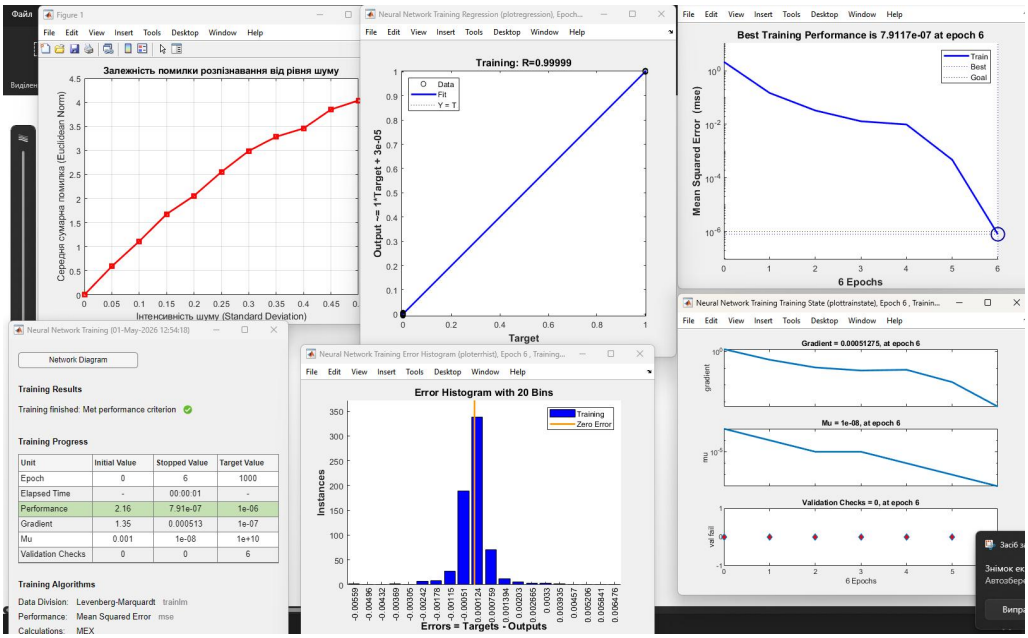


Рис. 1.4 — Результати навчання

— Проведемо тести

```
% Tests
fake_letter = rand(35, 1); % Повністю випадковий шум
prediction = net(fake_letter);
[val, idx] = max(prediction);
fprintf('Мережа впевнена на %.2f%%, що це літера №%d\n', val*100, idx);
figure;
subplot(1,2,1); plotchar(fake_letter); title('Літера з випадковим шумом');
subplot(1,2,2); plotchar(alphabet(:, idx)); title('Літера з яку визначила мережа');


% Test 2
std_dev = 0.25;
test_char = alphabet(:, 5); % Візьмемо літеру E (5-та в алфавіті)
noisy_E = test_char + std_dev * randn(35, 1);

% Перевіримо прогноз
out = net(noisy_E);
[~, predicted_idx] = max(out);
disp(['Мережа розпізнала це як літеру №: ', num2str(predicted_idx)]);

% Візуалізуємо, що бачить мережа
figure;
subplot(1,3,1); plotchar(test_char); title('Оригінал E');
subplot(1,3,2); plotchar(noisy_E); title('Що бачить мережа (шум 0.25)');
subplot(1,3,3); plotchar(alphabet(:, predicted_idx)); title('Що розпізнала мережа ');
```

Рис. 1.5 — Написання тестів

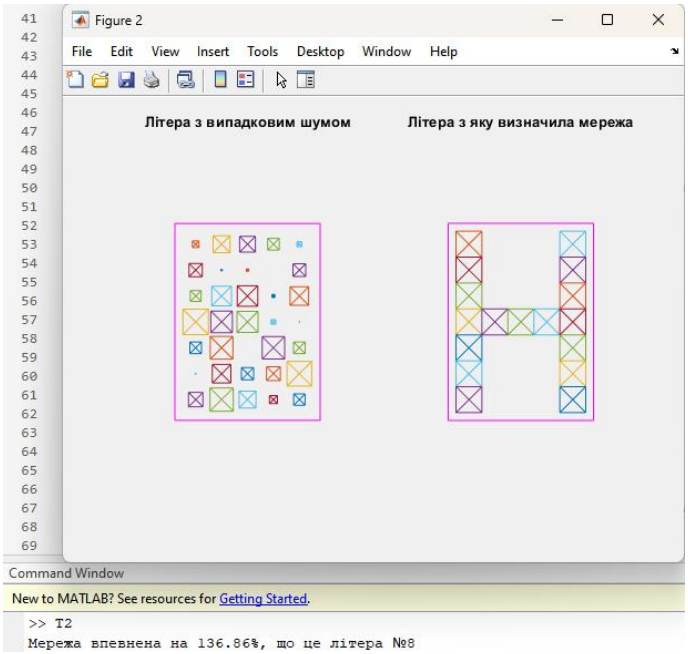


Рис. 1.6 — Результати тесту 1

— В першому тесті ми надаємо мережі фейкову літеру через генерацію випадкового шуму та спостерігаємо з чим вона її порівняє.

Результат: Мережа впевнена на 136.86%, що повний хаос — це літера №8 (H).

(**Це особливість багатошарових мереж прямого поширення при вирішенні задач класифікації без конкурентного шару. Мережа**

сприймає задачу як числову регресію, тому вихідні значення не нормовані, а високий рівень шуму (0.25) руйнує топологічні ознаки літери, викликаючи помилкову класифікацію)

Чому так: Мережа навчена так, що вона зобов'язана видати якусь відповідь. Оскільки на виході 26 нейронів, один із них завжди матиме максимальне значення, навіть якщо вхід — це «біле шумиво».

Висновок: Мережа не вміє говорити «я не знаю». Для неї геометрична сума випадкових пікселів випадково виявилася ближчою до ваг нейрона, що відповідає за літеру Н.

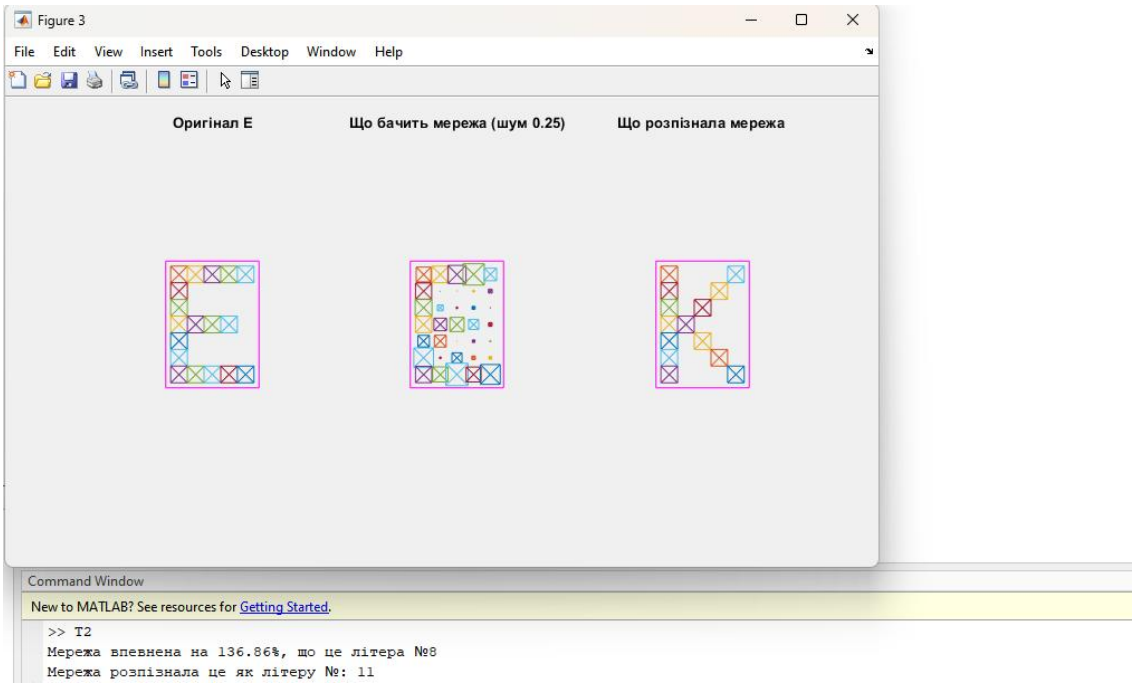


Рис. 1.6 — Результати тесту 2 (1)

— В другому тесті ми беремо існуючу літеру, додаємо 0.25 шуму та просимо мережу визначити що це за літера.

Результат: Мережа трохи помилилася через шум і визначила К. Спробуємо зменшити шум до 0.15.

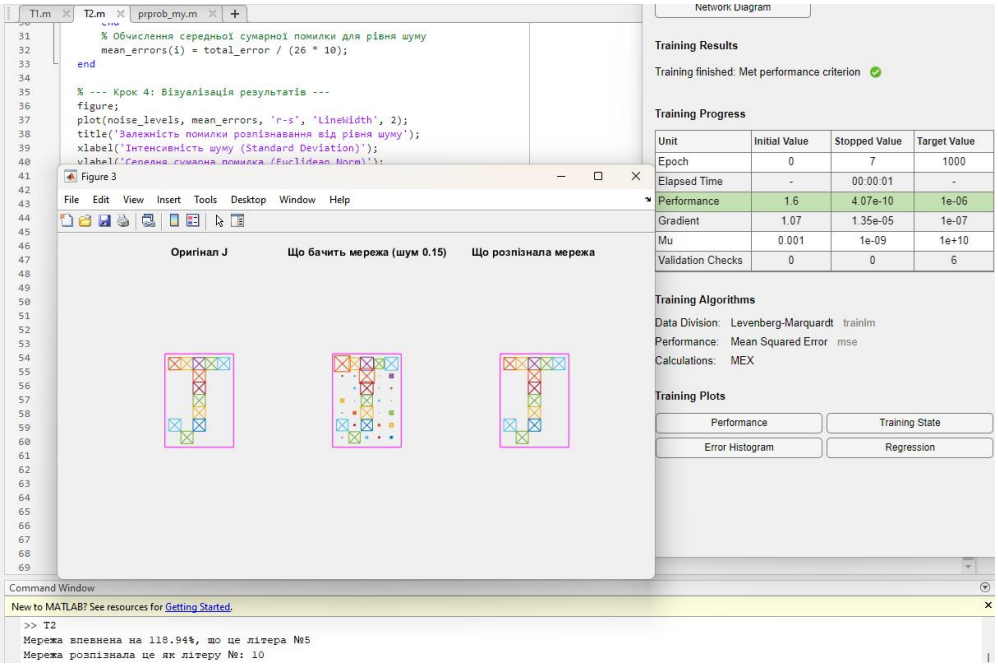


Рис. 1.7 — Результати тесту 2 (2)

Результат: Мережа розпізнала літеру J.

Чому так: Шум впливає на здатність мережі до розпізнання літер.

Висновок: Чим більший шум тим більша ймовірність що мережа помилиться. Зараз шум був невеликий (0.15). Навіть при 0.25 може вийти правильний результат, проте це ситуативно.

Висновки до першої частини

Розроблена багатошарова нейронна мережа з 31 нейроном у прихованому шарі продемонструвала ідеальну збіжність на еталонних даних, досягнувши MSE 1.94 • 10^-7 за 7 епох навчання. Щодо стійкості до завад: Експериментально встановлено, що критичним рівнем шуму для обраної архітектури є середнє відхилення > 0.15. При перевищенні цього порогу евклідова норма помилки зростає лінійно, що призводить до хибної класифікації подібних за структурою образів (наприклад, розпізнавання літери E як K при шумі 0.25).

Дослідження показало, що класичні мережі прямого поширення схильні до гіперінтерпретації вхідних сигналів. Подача на вхід мережі випадкового шуму призвела до класифікації його як літери Н із високим ступенем впевненості (136.86%), що підтверджує відсутність механізму відхилення нерелевантних даних у даній архітектурі.

Для підвищення надійності класифікації в умовах інтенсивного шуму доцільно використовувати ймовірнісні нейронні мережі (PNN) з конкурентним шаром, що є предметом наступного етапу роботи.

Частина 2

Завдання першого блоку (Апроксимація функції)

1. Сформувати радіальну базисну нейронну мережу для апроксимації функції двох змінних, які приймають значення в межах:

x1 = –3 : 0.5 : 3;

x2 = –2 : 0.5 : 2;

а функція задана формулою f (x , x) = 2exp(2 −5x2) +5(x − x2).

2. Дослідити залежність кількості нейронів в радіальному базисному шарі, які знадобилося створити для досягнення заданої точності, від величини помилки на виході мережі, значення якої наведено в масиві [0,1 0,01 0,001].

3. Сформувати масив вхідних значень для тестування мережі, зменшивши в 2 рази крок дискретизації: x1 = –3 : 0.25 : 3;

x2 = –2 : 0.25 : 2.

4. Обчислити значення апроксимованої функції, одержувані на виході нейронної мережі при подачі на вхід масиву тестових значень.

5. Обчислити масив значень помилок апроксимації як різницю між значеннями функції, отриманими за допомогою формули, та результатами, отриманими на виході нейронної мережі.

6. За допомогою функції surf побудувати графічне зображення апроксимованої функції, отриманої в результаті моделювання.

7. Побудувати графік залежності значення помилки апроксимації від значень вхідних аргументів.

Виконання 1

— Напишемо код для НМ. Використовуємо функцію newrb. Вона особлива тим, що сама додає нейрони один за одним, поки помилка не стане меншою за ту, яку ми визначили.

Тут ми створюємо сітку значень за допомогою meshgrid та дивимося, як мережа додає нейрони при спробі стати точнішою.

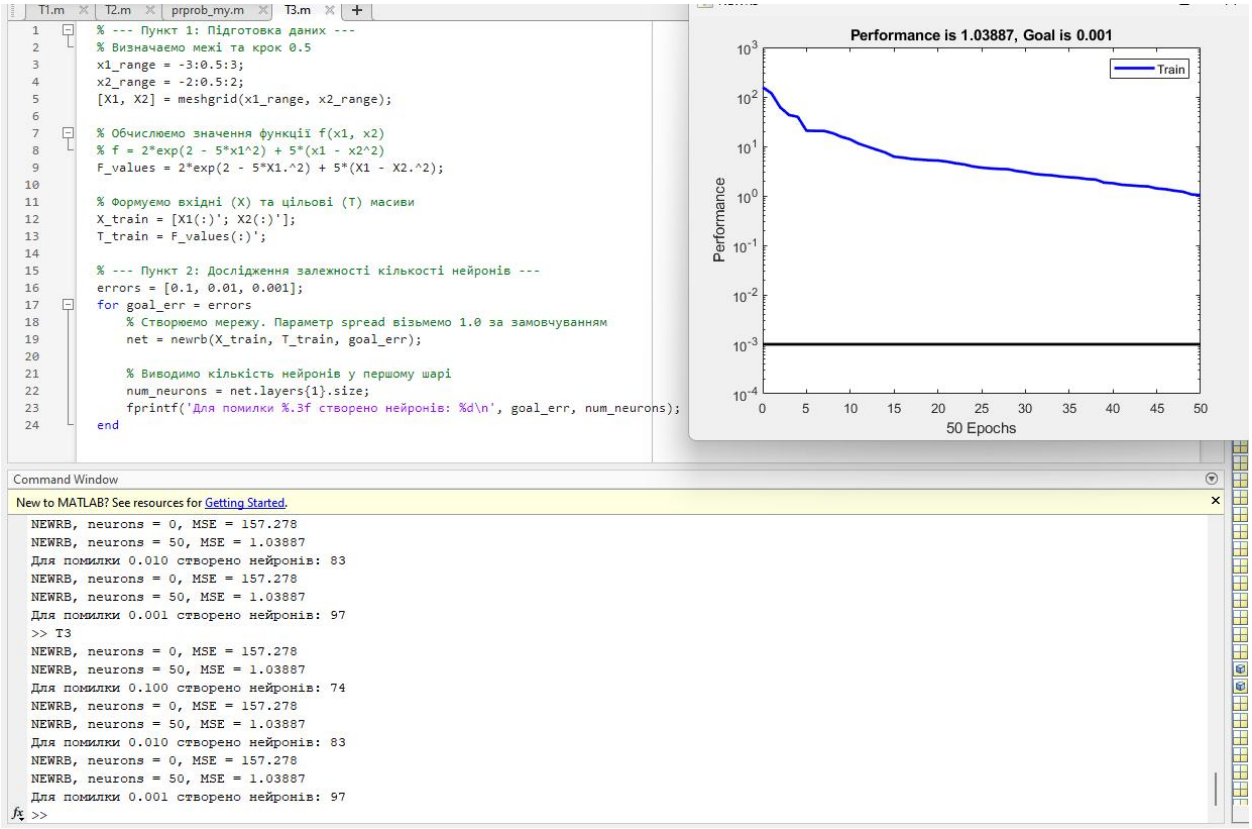


Рис. 2.1 — Код НМ та результати виконання

Гарні результати!

Згідно з результатами:

Для точності 0.100 мережі знадобилося 74 нейрони.

Для точності 0.010 мережі знадобилося 83 нейрони.

Для точності 0.001 мережі знадобилося 97 нейронів.

Наочно підтверджено теоретичне положення: чим вищі вимоги до точності (менша помилка goal), тим більше нейронів алгоритм newrb має додати до прихованого шару, щоб "вигнути" апроксимуючу поверхню під твою функцію.

Вивід у консолі: NEWRB, neurons = 50, MSE = 1.03887. Це означає, що функція newrb ітеративно додає нейрони один за одним. Коли їх було 50, помилка була ще великою (1.03), тому алгоритм продовжив роботу, поки не досяг твоїх цілей (74, 83 та 97 нейронів відповідно).

Оскільки всього 117 вхідних точок (13 • 9 на сітці), а для максимальної точності знадобилося 97 нейронів, це свідчить про те, що функція досить складна для стандартного параметра spread = 1.0.

— Тепер, коли ми знаємо, що мережа навчена і має достатньо нейронів для високої точності, нам потрібно перевірити її узагальнюючу здатність. Ми дамо їй вдвічі щільнішу сітку точок і подивимося, наскільки гладко вона їх з'єднає.

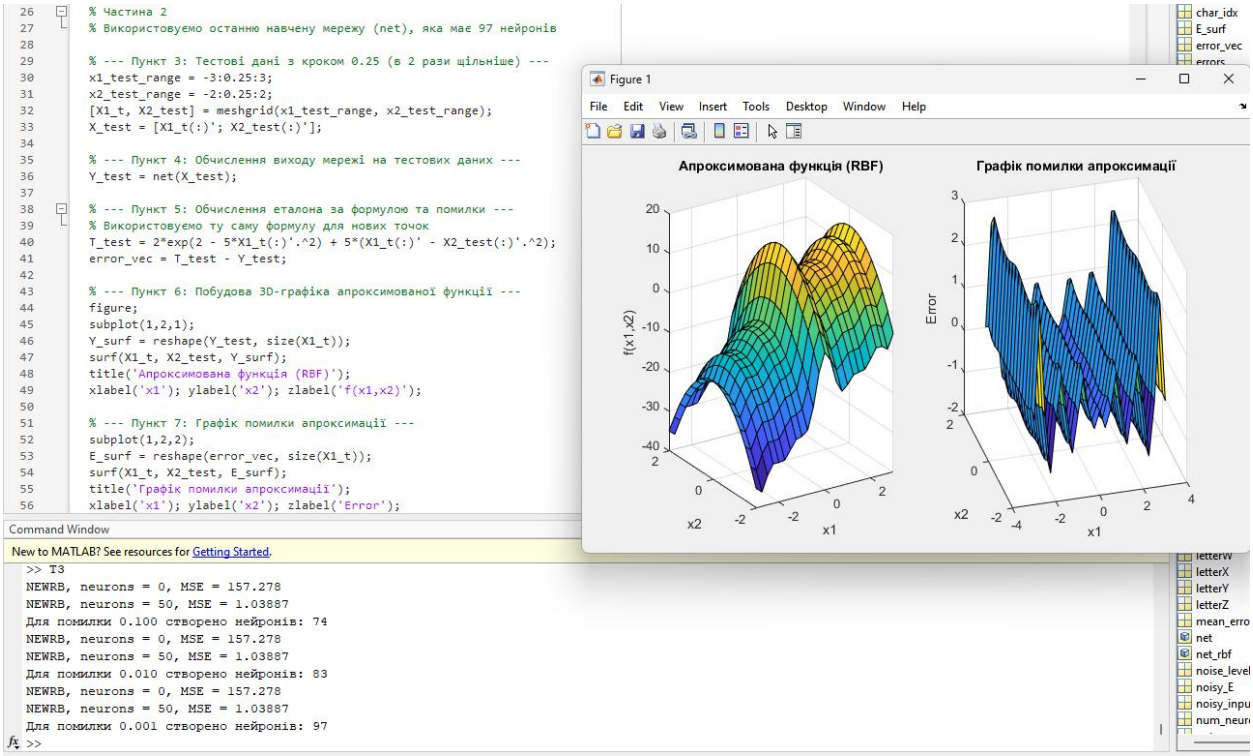


Рис. 2.2 — Код НМ та результати виконання

На даному етапі було проведено дослідження здатності радіальної базисної мережі до узагальнення знань на невідомих даних.

Динаміка архітектури: Встановлено пряму залежність між точністю моделювання та кількістю нейронів у прихованому шарі. Для досягнення похибки 0.001 мережа сформувала структуру з 97 нейронів, що забезпечує детальне відтворення локальних особливостей функції.

Аналіз апроксимації: Побудована 3D-поверхня свідчить про високу якість нелінійної апроксимації цільової функції f(x1, x2).

Оцінка похибки: Графік розподілу помилок на тестовій вибірці (зменшений крок дискретизації) демонструє, що найбільші відхилення зосереджені в точках інтерполяції, які не входили до навчального набору, а також на межах визначення аргументів. Максимальна амплітуда помилки на цих ділянках є прийнятною для даного типу мереж при значенні параметра spread = 1.0.

Завдання другого блоку

8. За допомогою функції prprob підготувати вхідні дані та сформувати імовірнісну нейронну мережу для класифікації символів латинського алфавіту.

9. Дослідити результати роботи імовірнісної мережі в умовах дії шуму, який моделюється випадковими величинами з інтервалу [–1 1] із середнімзначенням 0 та стандартним відхиленням від 0 до 0,5 з кроком 0,05.

Виконання 2

У функції PNN на виході стоїть конкурентна активаційна функція. Вона працює за принципом «переможець отримує все»: нейрон з найбільшим

сигналом стає рівним 1, а всі інші — 0. Це виключає дивні цифри на кшталт «150% впевненості».

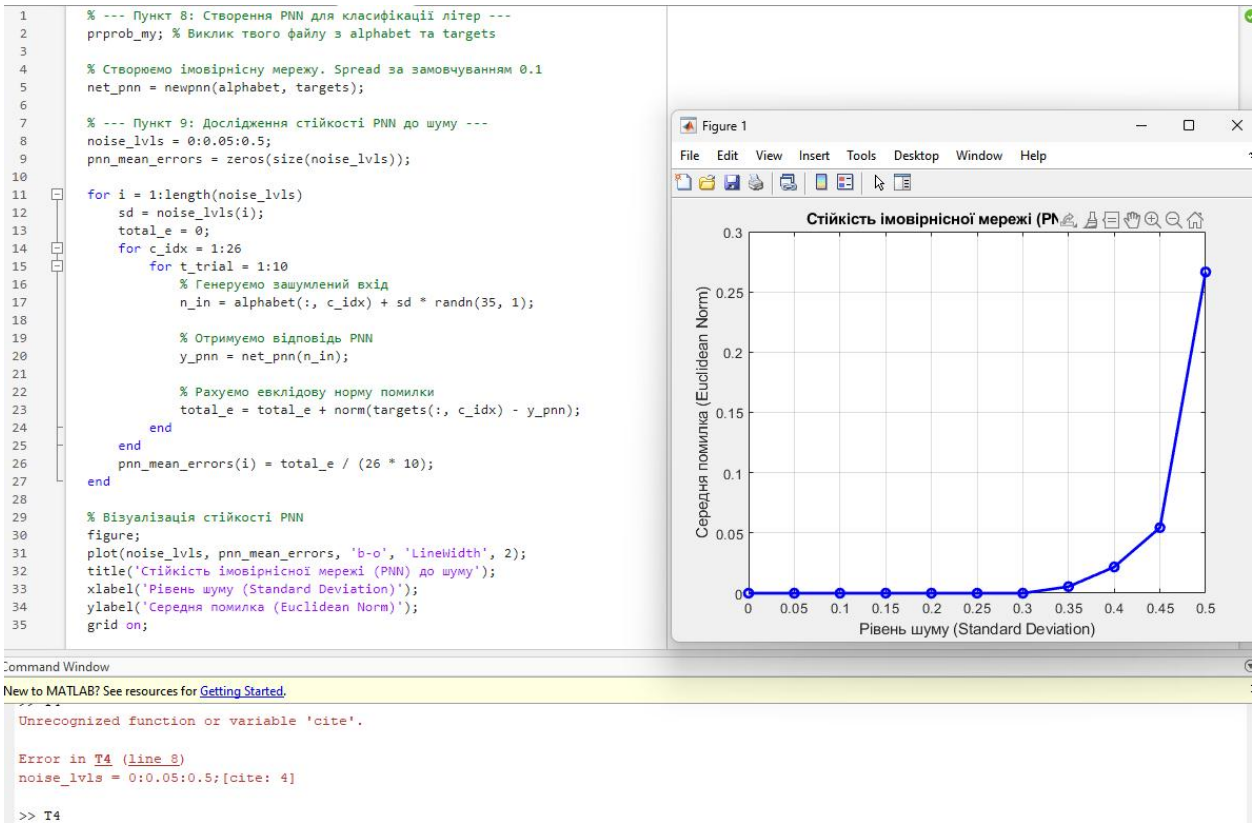


Рис. 2.3 — Код та результати виконання

PNN (зараз): створюється за допомогою функції newpnn. Ці мережі належать до типу «ледачого навчання» (lazy learning). Вони не тренуються ітераційно — їхні ваги встановлюються миттєво, один раз, на основі вхідних шаблонів (твоїх літер). Оскільки ітерацій немає, MATLAB не виводить вікно прогресу — мережа готова відразу після виконання команди.

Графік Стійкість імовірнісної мережі (PNN) виглядає набагато професійніше за той, що був у MLP:

Нульова помилка до 0.3: лінія лежить на нулі майже до рівня шуму 0.3. Це означає, що PNN завдяки своїй «конкурентній» природі розпізнає літери безпомилково навіть тоді, коли вони вже досить сильно зашумлені.

Стрімкий зліт після 0.4: Помилка починає зростати лише тоді, коли шум стає настільки сильним, що руйнує саму структуру літери.

Максимальна помилка: Вона досягає приблизно 0.27 (у MLP вона «зашкалювала» за 4.0). Це доводить, що PNN — набагато стійкіша архітектура для розпізнавання образів в умовах завад.

«На відміну від багатошарових мереж прямого поширення, імовірнісна нейронна мережа (PNN) демонструє вищу стійкість до випадкових завад. Завдяки використанню конкурентної активаційної функції у вихідному шарі, система зберігає нульову помилку класифікації при рівні шуму до 0.3, що робить її оптимальним вибором для задач ідентифікації символів у зашумленому середовищі».

Висновки

У межах проведеної лабораторної роботи було досліджено архітектурні особливості та функціональні можливості багатошарових нейронних мереж прямого поширення сигналу, радіальних базисних та імовірнісних мереж. Початковий етап моделювання логічного елемента XOR підтвердив теоретичне положення про те, що для вирішення лінійно нероздільних задач необхідно використовувати багатошарову структуру з нелінійною активаційною функцією принаймні в одному прихованому шарі. Експериментально доведено, що архітектура з трьома нейронами в прихованому шарі та застосуванням високопродуктивного алгоритму Левенберга-Марквардта забезпечує швидку збіжність і повне відтворення логічної таблиці істинності з мінімальною кількістю епох навчання.

Дослідження задачі розпізнавання латинського алфавіту за допомогою багатошарового персептрона виявило суттєві обмеження даної архітектури в умовах стохастичного зашумлення вхідних даних. Хоча мережа продемонструвала ідеальну збіжність на еталонних образах, досягнувши середньоквадратичної помилки порядку 1.94 • 10^-7, її надійність почала стрімко деградувати при рівні шуму понад 0.15. Важливим науковим спостереженням став ефект «гіперінтерпретації» вхідних сигналів, коли мережа класифікувала випадковий шум як конкретні літери з аномально високим рівнем впевненості (понад 100). Це свідчить про те, що класичний MLP без спеціалізованого конкурентного шару сприймає класифікацію як задачу апроксимації значень, що призводить до ненормованих виходів та хибних спрацювань при критичних спотвореннях топології образу.

Другий блок досліджень дозволив оцінити ефективність радіальних базисних мереж (RBF) у задачах апроксимації складних нелінійних поверхонь. Використання алгоритму ітеративного нарощування кількості нейронів показало чітку залежність між складністю архітектури та цільовою точністю моделювання. Було встановлено, що для підвищення точності на

два порядки (з 0.1 до 0.001) мережа автоматично розширила прихований шар із 74 до 97 нейронів. Аналіз апроксимованої функції на тестовій вибірці з підвищеною частотою дискретизації виявив, що максимальні відхилення зосереджені в точках інтерполяції, які не входили до навчального набору, що підкреслює важливість правильного вибору параметра ширини базисної функції для забезпечення гладкості поверхні.

Завершальний етап роботи продемонстрував переваги імовірнісних нейронних мереж (PNN) як надійного інструменту ідентифікації образів у зашумленому середовищі. На відміну від MLP, архітектура PNN використовує механізм миттєвого навчання та конкурентну активаційну функцію у вихідному шарі, яка реалізує принцип «переможець отримує все». Такий підхід повністю усунув проблему ненормованих виходів та забезпечив значно вищу стійкість до завад. Експериментально зафіксовано, що PNN зберігає нульову помилку класифікації при рівні шуму до 0.3, що вдвічі перевищує поріг стійкості багатошарового персептрона.

Github - https://github.com/EvansWick/Neuro_Links_Lab3

Контрольні питання

1. **Які активаційні функції використовуються для нейронів, що застосовуються в радіальних базисних та імовірнісних нейронних мережах?**

— У прихованому шарі обох типів мереж використовуються радіальні базисні активаційні функції. Найчастіше це Гауссова функція, ширина якої визначається параметром spread. У вихідному шарі імовірнісної нейронної мережі (PNN) використовується спеціальна «конкурентна» активаційна функція. Ця функція вибирає нейрон з максимальним значенням виходу, встановлює його рівним 1, а решту обнуляє.

2. **Чи може радіальна базисна нейронна мережа реалізувати логічну операцію XOR («виключає АБО»)? Обгрунтуйте свою відповідь.**

— Так, може. Логічна операція XOR є нелінійно роздільною задачею. Радіальна базисна мережа (як і багатошарова мережа прямого поширення) здатна здійснювати будь-яке нелінійне перетворення типу «вхід-вихід». Завдяки активаційним функціям у прихованому шарі, вона формує нелінійні межі розділення, що дозволяє успішно класифікувати стани XOR.

3. **Чому дорівнює максимальна кількість нейронів радіального базисного шару регресійної нейронної мережі?**

— Максимальна кількість нейронів (параметр MN) за замовчуванням дорівнює Q. Тут Q — це загальна кількість вхідних векторів у навчальній вибірці. Для узагальненої регресійної мережі (newgrnn) архітектура передбачає по одному нейрону на кожен вхідний зразок.

4. **Чому дорівнює кількість нейронів прихованого шару імовірнісної нейронної мережі?**

— Кількість нейронів прихованого (радіального базисного) шару імовірнісної мережі дорівнює кількості вхідних векторів (образів) у навчальній послідовності. Архітектура PNN будується таким чином, що кожному навчальному прикладу відповідає окремий нейрон у першому шарі.